

Introducción a la Programación I

Clase 9: Archivos y Excepciones

Agenda

- **Archivos**

- ¿Qué es un archivo y por qué los necesitamos?
- Tipos de archivos: `.txt`, `.csv`, binarios, `.zip`
- Trabajar con archivos en Python
- Operaciones comunes

- **Excepciones**

- ¿Qué es una excepción?
- Sintaxis: `try`, `except`, `else`, `finally`
- Capturar y lanzar excepciones
- Excepciones built-in y buenas prácticas

¿Qué es un archivo?

Un **archivo** es una colección de datos almacenados en el sistema de almacenamiento de una computadora (disco rígido, SSD, etc.).

- Puede contener distintos tipos de datos: texto, números, imágenes, videos o información binaria.
- Es accesible por los programas para leer, escribir o manipular su contenido.

Ejemplos:

<code>notas.txt</code>	→ archivo de texto
<code>datos.csv</code>	→ tabla de datos
<code>foto.jpg</code>	→ imagen
<code>programa.exe</code>	→ ejecutable

¿Por qué necesitamos archivos?

-  **Almacenamiento persistente:** los datos sobreviven al cierre del programa o al apagado de la computadora. Sin archivos, todo se pierde en memoria RAM.
-  **Compartir datos:** los archivos pueden enviarse entre programas o sistemas (por email, red, etc.).
-  **Organizar información:** permiten categorizar y estructurar datos (documentos en carpetas, logs, datasets).

Características de los archivos

Nombre y extensión

Todo archivo tiene un **nombre** y una **extensión** que indica su tipo:

```
notas.txt    → "notas" es el nombre, ".txt" es la extensión  
datos.csv   → "datos" es el nombre, ".csv" es la extensión
```

Ruta (path)

La **ruta** describe la ubicación del archivo en el sistema:

```
Windows:  C:/Documentos/notas.txt  
Linux/Mac: /home/usuario/documentos/notas.txt
```

La ruta puede ser **absoluta** (desde la raíz) o **relativa** (desde el directorio actual).

Tipos de archivos: `.txt`

Los **archivos de texto plano** contienen datos legibles por humanos.

- Usados para notas, logs, configuraciones, etc.
- Se pueden abrir con cualquier editor de texto.

Ejemplo: `lista_compras.txt`

```
Manzanas  
Bananas  
Pan  
Leche
```

Trabajar con archivos `.txt` en Python

```
# Leer un archivo
with open('ejemplo.txt', 'r') as archivo:
    contenido = archivo.read()
    print(contenido)
# Escribir en un archivo
with open('ejemplo.txt', 'w') as archivo:
    archivo.write("Hola, esto es un texto de ejemplo.")
# Agregar contenido al final
with open('ejemplo.txt', 'a') as archivo:
    archivo.write("\nUna línea más.")
```

Modos de apertura de archivos

Modo	Descripción
'r'	Lectura (el archivo debe existir)
'w'	Escritura (crea o sobrescribe)
'a'	Agregar al final (sin sobrescribir)

El bloque `with` — Contexto de archivo

El bloque `with` se usa para abrir archivos de forma **segura**:

```
with open('notas.txt', 'r') as archivo:
    contenido = archivo.read()
    print(contenido)
# El archivo se cierra automáticamente al salir del bloque
```

✓ **Ventaja:** el archivo se cierra **automáticamente** aunque ocurra un error dentro del bloque. No hace falta llamar a `archivo.close()` manualmente.

Tipos de archivos: **.csv**

Los archivos **CSV (Comma-Separated Values)** almacenan datos estructurados como tablas.

- Cada fila es un registro.
- Las columnas están separadas por comas.

Ejemplo: `datos.csv`

```
Nombre, Edad, Nota  
Alice, 20, B  
Bob, 19, A  
Carlos, 21, C
```

Trabajar con archivos **.csv** en Python

```
import csv

# Leer un CSV
with open('datos.csv', mode='r') as archivo:
    lector = csv.reader(archivo)
    for fila in lector:
        print(fila)

# Escribir un CSV
with open('datos.csv', mode='w', newline='') as archivo:
    escritor = csv.writer(archivo)
    escritor.writerow(['Nombre', 'Edad', 'Nota'])
    escritor.writerow(['Alice', 20, 'B'])
    escritor.writerow(['Bob', 19, 'A'])
```

Tipos de archivos: binarios

Los **archivos binarios** almacenan datos en formato de 0s y 1s; no son legibles por humanos directamente.

Ejemplos:

foto.jpg → imagen
cancion.mp3 → audio
programa.exe → ejecutable

- Requieren programas específicos para abrirse e interpretarse.
- En Python se abren con modo `'rb'` (read binary) o `'wb'` (write binary).

```
with open('imagen.jpg', 'rb') as archivo:  
    datos = archivo.read()  
    print(f"Tamaño: {len(datos)} bytes")
```

Tipos de archivos: **.zip**

Los archivos **ZIP** contienen uno o más archivos/carpetas **comprimidos**.

- Reducen el tamaño de los archivos.
- Son convenientes para compartir múltiples archivos juntos.

Ejemplo: proyecto.zip puede contener:

- notas.txt
- datos.csv
- foto.jpg

Trabajar con archivos `.zip` en Python

```
import zipfile

# Comprimir archivos
with zipfile.ZipFile('archivo.zip', 'w') as zip_arch:
    zip_arch.write('ejemplo.txt')
    zip_arch.write('datos.csv')

# Extraer archivos de un zip
with zipfile.ZipFile('archivo.zip', 'r') as zip_arch:
    zip_arch.extractall('archivos_extraidos/')

# Listar archivos dentro del zip
with zipfile.ZipFile('archivo.zip', 'r') as zip_arch:
    print(zip_arch.namelist())
# ['ejemplo.txt', 'datos.csv']
```

Operaciones comunes con archivos

```
# Leer el archivo completo de una vez
with open('ejemplo.txt', 'r') as archivo:
    contenido = archivo.read()
    print(contenido)

# Leer línea por línea
with open('ejemplo.txt', 'r') as archivo:
    for linea in archivo:
        print(linea.strip()) # .strip() elimina el \n al final

# Verificar si un archivo existe
import os
if os.path.exists('ejemplo.txt'):
    print("El archivo existe")
else:
    print("Archivo no encontrado")
```

Excepciones — ¿Qué es una excepción?

Una **excepción** es un evento que **interrumpe el flujo normal** de ejecución de un programa.

- Ocurre cuando se produce un error en tiempo de ejecución (ej: dividir por cero, acceder a un índice inválido).
- El manejo de excepciones permite:
 - Detectar y responder a errores de forma **controlada**.
 - **Evitar** que el programa termine abruptamente.
 - Separar la lógica principal del código de manejo de errores.

¿Por qué necesitamos el manejo de excepciones?

-  **Robustez:** el programa puede seguir ejecutándose aunque ocurra un error inesperado.
-  **Claridad:** el código de manejo de errores (`except`) está separado del código "feliz" (`try`).
-  **Propagación de errores:** si una función no puede manejar una excepción, la deja "subir" al código que la llamó.
-  **Mantenibilidad:** centralizar el manejo de errores facilita la detección y corrección de problemas.

Sintaxis básica: `try` / `except`

```
try:
    # Bloque protegido: código que puede fallar
    resultado = 10 / divisor
    print("El resultado es:", resultado)
except ZeroDivisionError:
    # Se ejecuta si ocurre un ZeroDivisionError
    print("Error: no se puede dividir por cero.")
```

- El código dentro de `try` se ejecuta normalmente.
- Si ocurre una excepción del tipo indicado en `except`, se ejecuta ese bloque.
- Si no ocurre ninguna excepción, el bloque `except` se **omite**.

Bloques `else` y `finally`

```
try:
    archivo = open("datos.txt", "r")
    contenido = archivo.read()
except FileNotFoundError:
    print("Error: el archivo no existe.")
else:
    # Solo se ejecuta si NO ocurrió ninguna excepción
    print("Contenido del archivo:")
    print(contenido)
finally:
    # Siempre se ejecuta, haya o no excepción
    print("Operación finalizada.")
```

Bloque	¿Cuándo se ejecuta?
<code>else</code>	Solo si <code>try</code> no lanzó excepción
<code>finally</code>	Siempre , haya o no excepción

Capturar múltiples excepciones

```
try:
    valor = int(input("Ingresa un número: "))
    resultado = 100 / valor
except ValueError:
    print("Error: debes ingresar un número entero válido.")
except ZeroDivisionError:
    print("Error: no se puede dividir por cero.")
except Exception as e:
    # Captura cualquier otra excepción inesperada
    print("Ocurrió un error inesperado:", e)
else:
    print("Cálculo exitoso. Resultado:", resultado)
```

También se pueden agrupar excepciones en un mismo bloque:

```
except (ValueError, TypeError):
    print("Error de tipo o valor.")
```

El bloque genérico `Exception as e`

```
try:
    numero = int("veinte")
except ValueError as error:
    print("Capturé un ValueError:", error)
    print("Tipo de excepción:", type(error).__name__)
```

```
Capturé un ValueError: invalid literal for int() with base 10: 'veinte'
Tipo de excepción: ValueError
```

- `error.args` contiene una tupla con los argumentos con los que se lanzó la excepción.
- `type(error).__name__` devuelve el nombre del tipo de excepción.

Lanzar excepciones con `raise`

A veces queremos **generar una excepción explícitamente** cuando detectamos una condición inválida:

```
def calcular_raiz(x):  
    if x < 0:  
        raise ValueError("No se puede calcular la raíz de un número negativo.")  
    return x ** 0.5  
  
try:  
    resultado = calcular_raiz(-4)  
    print("Resultado:", resultado)  
except ValueError as ve:  
    print("Error en calcular_raiz:", ve)  
# → Error en calcular_raiz: No se puede calcular la raíz de un número negativo.
```

`raise` permite **validar argumentos** y señalar condiciones de error dentro de funciones.

Excepciones built-in más comunes

Excepción	Cuándo ocurre
<code>ZeroDivisionError</code>	División por cero
<code>ValueError</code>	Valor inválido (ej: <code>int("abc")</code>)
<code>TypeError</code>	Tipo incorrecto (ej: <code>"a" + 1</code>)
<code>IndexError</code>	Índice fuera de rango en lista/tupla
<code>KeyError</code>	Clave inexistente en diccionario
<code>FileNotFoundError</code>	Archivo o directorio no encontrado
<code>IOError</code> / <code>OSError</code>	Errores de entrada/salida del sistema
<code>ArithmeticError</code>	Clase base para errores aritméticos

Ejemplos de excepciones built-in

```
# IndexError
mi_lista = [1, 2, 3]
try:
    print(mi_lista[5])
except IndexError:
    print("¡El índice está fuera de rango!")

# FileNotFoundError
try:
    with open("no_existe.txt", "r") as f:
        contenido = f.read()
except FileNotFoundError:
    print("¡El archivo no fue encontrado!")
```


Inspeccionar información de la excepción

```
try:
    numero = int("veinte")
except ValueError as error:
    print("Capturé ValueError:", error)
    print("Tipo:", type(error).__name__)
    print("Args:", error.args)
```

Para obtener el **traceback** completo (secuencia de llamadas que llevó al error):

```
import traceback

try:
    1 / 0
except ZeroDivisionError:
    traceback.print_exc()
```

Buenas prácticas en el manejo de excepciones

1. **No usar excepciones para el flujo normal** — solo para situaciones inesperadas o errores.
2. **Capturar la excepción más específica posible** — evitar `except Exception:` salvo que sea necesario.
3. **Nunca "silenciar" excepciones sin razón** — un `except: pass` vacío dificulta el debugging.
4. **Liberar recursos en `finally` o usar context managers** — archivos, conexiones, sockets, etc.

```
with open("archivo.txt", "r") as f:  
    contenido = f.read()  
# No es necesario cerrar manualmente
```

Ejemplo práctico: pedir un entero al usuario

```
def pedir_entero(mensaje):  
    while True:  
        try:  
            valor = int(input(mensaje))  
            return valor  
        except ValueError:  
            print("Entrada inválida. Por favor ingresá un número entero.")  
  
edad = pedir_entero("Ingresá tu edad: ")  
print(f"Tu edad es: {edad}")
```

- Repite el pedido hasta que el usuario ingrese un entero válido.
- Evita que el programa se caiga por un `ValueError`.

Archivos + Excepciones: combinación práctica

```
import os

def leer_archivo(ruta):
    if not os.path.exists(ruta):
        raise FileNotFoundError(f"El archivo '{ruta}' no existe.")
    try:
        with open(ruta, 'r') as archivo:
            return archivo.read()
    except IOError as e:
        print(f"Error al leer el archivo: {e}")
        return None

contenido = leer_archivo("datos.txt")
if contenido:
    print(contenido)
```

Combinando verificación con `os.path.exists()` y manejo de excepciones logramos código más robusto.

Resumen y conclusiones

Archivos

- `open(ruta, modo)` con modos `'r'`, `'w'`, `'a'`
- Siempre usar el bloque `with` para garantizar el cierre
- Para CSV: `import csv` → `csv.reader()` / `csv.writer()`
- Para ZIP: `import zipfile` → `ZipFile()` con modos `'r'` / `'w'`

Excepciones

- `try` / `except` / `else` / `finally`
- Capturar excepciones específicas (no genéricas)
- `raise` para lanzar excepciones propias
- Excepciones comunes: `ValueError`, `TypeError`, `IndexError`, `KeyError`,
`FileNotFoundError`